

Лабораторная работа № 12

Архитектура компьютера, раздел Операционные системы

Бондарь Т. В.

Российский университет дружбы народов, Москва, Россия

Информация

- Бондарь Татьяна Владимировна
- НКАбд-01-24, студ. билет №1132246711
- Российский университет дружбы народов
- https://github.com/tvbondar/study_2024-2025_os-intro

Изучить основы программирования в оболочке ОС UNIX/Linux. Научиться писать небольшие командные файлы.

Задание

1. Написать скрипт, который при запуске будет делать резервную копию самого себя (то есть файла, в котором содержится его исходный код) в другую директорию backup в вашем домашнем каталоге. При этом файл должен архивироваться одним из архиваторов на выбор zip, bzip2 или tar. Способ использования команд архивации необходимо узнать, изучив справку.
2. Написать пример командного файла, обрабатывающего любое произвольное число аргументов командной строки, в том числе превышающее десять. Например, скрипт может последовательно распечатывать значения всех переданных аргументов.
3. Написать командный файл — аналог команды ls (без использования самой этой команды и команды dir). Требуется, чтобы он выдавал информацию о нужном каталоге и выводил информацию о возможностях доступа к файлам этого каталога.
4. Написать командный файл, который получает в качестве аргумента командной строки формат файла (.txt, .doc, .jpg, .pdf и т.д.) и вычисляет количество таких файлов в указанной директории. Путь к директории также передаётся в виде аргумента ко-

Теоретическое введение

Командный процессор (командная оболочка, интерпретатор команд shell) — это программа, позволяющая пользователю взаимодействовать с операционной системой компьютера. В операционных системах типа UNIX/Linux наиболее часто используются следующие реализации командных оболочек: – оболочка Борна (Bourne shell или sh) — стандартная командная оболочка UNIX/Linux, содержащая базовый, но при этом полный набор функций; – C-оболочка (или csh) — надстройка на оболочкой Борна, использующая C-подобный синтаксис команд с возможностью сохранения истории выполнения команд; – оболочка Корна (или ksh) — напоминает оболочку C, но операторы управления программой совместимы с операторами оболочки Борна; – BASH — сокращение от Bourne Again Shell (опять оболочка Борна), в основе своей совмещает свойства оболочек C и Корна (разработка компании Free Software Foundation). POSIX (Portable Operating System Interface for Computer Environments) — набор стандартов описания интерфейсов взаимодействия операционной системы и прикладных программ. Стандарты POSIX разработаны комитетом IEEE (Institute of Electrical and Electronics Engineers) для обеспечения совместимости

Выполнение лабораторной работы

Выполнение лабораторной работы

1. Написать скрипт, который при запуске будет делать резервную копию самого себя (то есть файла, в котором содержится его исходный код) в другую директорию backup в вашем домашнем каталоге. При этом файл должен архивироваться одним из архиваторов на выбор zip, bzip2 или tar.

```
./..
/bib
/image
/pandoc
Makefile
lab12-1.sh
lab12-2.sh
lab12-3.sh
lab12-4.sh
report.md
```

```
tvbondar@fedora:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab12/report$ chmod +x lab12-1.sh
tvbondar@fedora:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab12/report$ mkdir backup
tvbondar@fedora:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab12/report$ bash lab12-1
bash: lab12-1: No such file or directory
tvbondar@fedora:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab12/report$ bash lab12-1.sh
tar: /home/tvbondar/backup/backup.tar: Cannot open: No such file or directory
tvbondar@fedora:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab12/report$ cd ~
tvbondar@fedora:~$ bash lab12-1.sh
tar: prog1.sh: Cannot stat: No such file or directory
tar: Exiting with failure status due to previous errors
tvbondar@fedora:~$ bash lab12-1.sh
lab12-1.sh
tvbondar@fedora:~$ █
```

Рис. 2: 2

```
#!/bin/bash  
tar -cvf ~/backup/backup.tar lab12-1.sh
```

Рис. 3: 3



backup.tar

Рис. 4: 4

2. Написать пример командного файла, обрабатывающего любое произвольное число аргументов командной строки, в том числе превышающее десять.



```
#!/bin/bash
for A in $*
do echo $A
done
```

```
tvbondar@fedora:~$ bash lab12-2.sh iloveyou/  
iloveyou/  
tvbondar@fedora:~$ bash lab12-1.sh jh oujh iuhjjkj ouihyfpihj iyt 65 tr5 678  
lab12-1.sh  
tvbondar@fedora:~$ bash lab12-2.sh jh oujh iuhjjkj ouihyfpihj iyt 65 tr5 678  
jh  
oujh  
iuhjjkj  
ouihyfpihj  
iyt  
65  
tr5  
678  
tvbondar@fedora:~$
```

Рис. 6: 6

3. Написать командный файл — аналог команды `ls` (без использования самой этой команды и команды `dir`). Требуется, чтобы он выдавал информацию о нужном каталоге и выводил информацию о возможностях доступа к файлам этого каталога.

```
#!/bin/bash
for A in *
do
    if test -d "$A"
    then
        echo "$A^ is a directory"
    else
        echo -n "$A: is a file and "
        if test -w $A
        then
            echo writeable
            if test -r $A
            then
                echo "readable"
            else
                echo "neither readable or writeable"
            fi
        fi
    fi
done
```



```
tvbondar@fedora:~$ bash lab12-3.sh iloveyou
2025-03-15 09:52-29.mp4: is a file and lab12-3.sh: line 9: test: 2025-03-15: binary operator expected
australia^ is a directory
backup^ is a directory
bin^ is a directory
conf.txt: is a file and writeable
readable
Desktop^ is a directory
Documents^ is a directory
Downloads^ is a directory
equiplist2: is a file and writeable
readable
feathers: is a file and writeable
readable
file.txt: is a file and writeable
readable
git-extended^ is a directory
github.io^ is a directory
gpg-key.pub: is a file and writeable
readable
lab11.sh: is a file and writeable
readable
lab11.sh~: is a file and writeable
readable
lab12-1.sh: is a file and writeable
readable
lab12-1.sh~: is a file and writeable
readable
lab12-2.sh: is a file and writeable
readable
lab12-3.sh: is a file and writeable
readable
lab12-4.sh: is a file and writeable
```

Рис. 8: 8

4. Написать командный файл, который получает в качестве аргумента командной строки формат файла (.txt, .doc, .jpg, .pdf и т.д.) и вычисляет количество таких файлов в указанной директории. Путь к директории также передаётся в виде аргумента командной строки.

```
#!/bin/bash
format=""
directory=""
echo "Напишите формат файла"
read format
echo "Напишите директорию"
read directory
find "${directory}" -name "*.${format}" -type f | wc -l
ls
```

Рис. 9: 9

```
tvbongar@fedora:~$ chmod +x Lab12-4.sh
tvbongar@fedora:~$ bash Lab12-4.sh
Напишите формат файла
txt
Напишите директорию
/home/tvbongar
40
'2025-09-15 09:52:29.ept'  bin      Documents  feathers  github.io  lab11.sh-  lab12-2.sh  LICENSE  morefun  OS      Public  Templates  work      orwell.mp4
australia  conf.txt  Downloads  file.txt  gpg-key.pub lab12-1.sh lab12-3.sh  may      Music    Pictures  reports  test.txt  xuperall.mp4  презентация11.mp4
backup     Desktop  equiplist2  git-extended  lab11.sh  lab12-1.sh-  lab12-4.sh  monthly  my_os    play      ski_places  Videos  a0611.mp4
```

Рис. 10: 10

Контрольные вопросы

Контрольные вопросы

1. Объясните понятие командной оболочки. Приведите примеры командных оболочек. Чем они отличаются?

Командные процессоры или оболочки - это программы, позволяющие пользователю взаимодействовать с компьютером. Их можно рассматривать как настоящие интерпретируемые языки, которые воспринимают команды пользователя и обрабатывают их. Поэтому командные процессоры также называют интерпретаторами команд. На языках оболочек можно писать программы и выполнять их подобно любым другим программам. UNIX обладает большим количеством оболочек. Наиболее популярными являются следующие четыре оболочки: –оболочка Борна (Bourne) - первоначальная командная оболочка UNIX: базовый, но полный набор функций; –C-оболочка - добавка университета Беркли к коллекции оболочек: она надстраивается над оболочкой Борна, используя C-подобный синтаксис команд, и сохраняет историю выполненных команд; –оболочка Корна - напоминает оболочку C, но операторы управления программой совместимы с операторами оболочки Борна; –BASH - сокращение от Bourne Again Shell (опять оболочка Борна) в основе

2. Что такое POSIX? POSIX (Portable Operating System Interface for Computer Environments)- интерфейс переносимой операционной системы для компьютерных сред. Представляет собой набор стандартов, подготовленных институтом инженеров по электронике и радиотехнике (IEEE), который определяет различные аспекты построения операционной системы. POSIX включает такие темы, как программный интерфейс, безопасность, работа с сетями и графический интерфейс. POSIX-совместимые оболочки являются будущим поколением оболочек UNIX и других ОС. Windows NT рекламируется как система, удовлетворяющая POSIX-стандартам. POSIX-совместимые оболочки разработаны на базе оболочки Корна; фонд бесплатного программного обеспечения (Free Software Foundation) работает над тем, чтобы и оболочку BASH сделать POSIX-совместимой.

3. Как определяются переменные и массивы в языке программирования `bash`? Командный процессор `bash` обеспечивает возможность использования переменных типа строка символов. Имена переменных могут быть выбраны пользователем. Пользователь имеет возможность присвоить переменной значение некоторой строки символов. Например, команда `mark=/usr/andy/bin` присваивает значение строки символов `/usr/andy/bin` переменной `mark` типа строка символов. Значение, присвоенное некоторой переменной, может быть впоследствии использовано. Для этого в соответствующем месте командной строки должно быть употреблено имя этой переменной, которому предшествует метасимвол. Например команда `{имя переменной}` например, использование команд `b=/tmp/andy-ls -l myfile > blsls/tmp/andy — ls, ls — l >bls` приведет к подстановке в командную строку значения переменной `bls`. Если переменной `bls` не было предварительно присвоено никакого значения, то ее значением является символ пробел. оболочка `bash` позволяет создание массивов. Для создания массива используется команда `set` с флагом `-A`. За флагом следует имя переменной, а затем список значений, разделенных пробелом. Например, `set -A states Delaware Michigan "New Jersey"` Далее можно сделать добавление в массив, например,

4. Каково назначение операторов `let` и `read`? Команда `let` является показателем того, что последующие аргументы представляют собой выражение, подлежащее вычислению. Простейшее выражение - это единичный терм (`term`), обычно целочисленный. Целые числа можно записывать как последовательность цифр или в любом базовом формате. Этот формат — `radix#number`, где `radix` (основание системы счисления) - любое число не более 26. Для большинства команд основания систем счисления это - 2 (двоичная), 8 (восьмеричная) и 16 (шестнадцатеричная). Простейшими математическими выражениями являются сложение (+), вычитание (-), умножение (*), целочисленное деление (/) и целочисленный остаток (%). Команда `let` берет два операнда и присваивает их переменной.

5. Какие арифметические операции можно применять в языке программирования bash?

Оператор Синтаксис Результат
!
!expr Если expr равно 0, возвращает 1; иначе 0
!= expr1 !=expr2 Если expr1 не равно expr2, возвращает 1; иначе 0
% expr1%expr2 Возвращает остаток от деления expr1 на expr2
%= var=%expr Присваивает остаток от деления var на expr переменной var
&
expr1&expr2 Возвращает побитовое AND выражений expr1 и expr2
&& expr1&&expr2 Если и expr1 и expr2 не равны нулю, возвращает 1; иначе 0
&= var &= expr Присваивает var побитовое AND переменных var и выражения expr
*
expr1 * expr2 Умножает expr1 на expr2
= var = expr Умножает expr на значение var и присваивает результат переменной var
+
expr1 + expr2 Складывает expr1 и expr2
+= var += expr Складывает expr со значением var и результат присваивает var
-
expr Операция отрицания expr (называется унарный минус)
- expr1 - expr2 Вычитает expr2 из expr1
-= var -= expr Вычитает expr из значения var и присваивает результат var
/
expr1 / expr2 Делит expr1 на expr2
/= var /= expr Делит var на expr и присваивает результат var
<
expr1 < expr2 Если expr1 меньше, чем expr2, возвращает 1, иначе возвращает 0
«
expr1« expr2 Сдвигает expr1 влево на expr2 бит
«= var «= expr Побитовый сдвиг влево значения var на expr
<= expr1 <= expr2 Если expr1 меньше, или равно expr2, возвращает 1; иначе возвращает 0
=
var = expr Присваивает значение expr переменной var
==
expr1==expr2 Если expr1 равно expr2, возвращает 1; иначе

6. Что означает операция (())? Условия оболочки bash.
7. Какие стандартные имена переменных Вам известны? Имя переменной (идентификатор) — это строка символов, которая отличает эту переменную от других объектов программы (идентифицирует переменную в программе). При задании имен переменным нужно соблюдать следующие правила: \$ первым символом имени должна быть буква. Остальные символы — буквы и цифры (прописные и строчные буквы различаются). Можно использовать символ «_»; \$ в имени нельзя использовать символ «.»; \$ число символов в имени не должно превышать 255; \$ имя переменной не должно совпадать с зарезервированными (служебными) словами языка. Var1, PATH, trash, mon, day, PS1, PS2 Другие стандартные переменные: -HOME — имя домашнего каталога пользователя. Если команда cd вводится без аргументов, то происходит переход в каталог, указан- ный в этой переменной . -IFS — последовательность символов, являющихся разделителями в командной строке. Это символы пробел, табуляция и перевод строки(new line). -MAIL — командный процессор каждый раз перед выводом на экран промптера проверяет содержимое файла, имя которого указано в этой переменной, и если содержимое этого файла изменилось с момента последнего ввода

8. Что такое метасимволы? Такие символы, как ' < > * ? | " & являются метасимволами и имеют для командного процессора специальный смысл.
9. Как экранировать метасимволы? Снятие специального смысла с метасимвола называется экранированием метасимвола. Экранирование может быть осуществлено с помощью предшествующего метасимволу символа, который, в свою очередь, является метасимволом. Для экранирования группы метасимволов, ее нужно заключить в одинарные кавычки. Строка, заключенная в двойные кавычки, экранирует все метасимволы, кроме \$, ' , , ". Например, `-echo` выведет на экран символ, `-echo ab'|'cd` выдаст строку `ab/cd`.

10. Как создавать и запускать командные файлы? Последовательность команд может быть помещена в текстовый файл. Такой файл называется командным. `bash` командный_файл [аргументы] . `chmod +x имя_файла` Теперь можно вызывать свой командный файл на выполнение просто, вводя его имя с терминала так, как будто он является выполняемой программой. Командный процессор распознает, что в Вашем файле на самом деле хранится не выполняемая программа, а программа, написанная на языке программирования оболочки, и осуществит ее интерпретацию.

11. Как определяются функции в языке программирования `bash`? Группу команд можно объединить в функцию. Для этого существует ключевое слово `function`, после которого следует имя функции и список команд, заключенных в фигурные скобки. Удалить функцию можно с помощью команды `unset` с флагом `-f`. Команда `typeset` имеет четыре опции для работы с функциями: `-f` — перечисляет определенные на текущий момент функции; `--ft` — при последующем вызове функции иницирует ее трассировку; `--fx` — экспортирует все перечисленные функции в любые дочерние программы оболочек; `--fu` — обозначает указанные функции как автоматически загружаемые. Автоматически загружаемые функции хранятся в командных файлах, а при их вызове оболочка просматривает переменную `FPATH`, отыскивая файл с одноименными именами функций, загружает его и вызывает эти функции.

12. Каким образом можно выяснить, является файл каталогом или обычным файлом? `ls -ltr`
Если есть `d`, то является файл каталогом

13. Каково назначение команд `set`, `typeset` и `unset`? Используется команда `set` с флагом `-A`. За флагом следует имя переменной, а затем список значений, разделенных пробелом. Например, `set -A states Delaware Michigan "New Jersey"`. Далее можно сделать добавление в массив, например, `states[49]=Alaska`. Индексация массивов начинается с нулевого элемента. В командном процессоре Си имеется еще несколько стандартных переменных. Значение всех переменных можно просмотреть с помощью команды `set`. Наиболее распространенным является сокращение, избавляющееся от слова `let` в программах оболочек. Если объявить переменные целыми значениями, любое присвоение автоматически трактуется как арифметическое. Используйте `typeset -i` для объявления и присвоения переменной, и при последующем использовании она становится целой. Или можете использовать ключевое слово `integer` (псевдоним для `typeset -l`) и объявлять переменные целыми. Таким образом, выражения типа `x=y+z` воспринимаются как арифметические. Группу команд можно объединить в функцию. Для этого существует ключевое слово `function`, после которого следует имя функции и список команд, заключенных в фигурные скобки. Удалить функцию можно с помощью команды `unset` с флагом `-f`. Команда `typeset` имеет четыре опции для работы с функциями: – `-f` —

14. Как передаются параметры в командные файлы? Символ \$ является метасимволом командного процессора. Он используется, в частности, для ссылки на параметры, точнее, для получения их значений в командном файле.

15. Назовите специальные переменные языка bash и их назначение.

\$* — отображается вся командная строка или параметры оболочки;

\$? — код завершения последней выполненной команды;

\$\$ — уникальный идентификатор процесса, в рамках которого выполняется командный процессор;

#! — номер процесса, в рамках которого выполняется последняя вызванная на выполнение в командном режиме команда;

\$- — значение флагов командного процессора;

и др.

Выводы

Мы изучили основы программирования в оболочке ОС UNIX/Linux. Научились писать небольшие командные файлы.

Список литературы
